

INF201 - Lecture 5

5th Oct 2026

Topics

- Extra assignment this week
- Deliverables
- Jupyter notebooks
- Collaborative workflows using GitHub
- Getting data from external sources
- Recursive functions
- Character encodings

Extra assignment for week 41

- There will be one extra assignment for this week
- For students missing points from the first four weeks
- The same rules as earlier
 - Mandatory attendance
 - Hand in code on Canvas by Friday at 16
 - Max 5 points
- Please let the teaching assistant know that you are taking this assignment, so they note down your attendance. This is your own responsibility!

Deliverables

- We start working on the first deliverable this week
- Deliverables are projects you work on in pairs, and submit every other week
- You have four deliverables in total, to be submitted by Friday at 16.00 in week 42, 44, 46 and 48
- You get up to 10 points per deliverable
- To pass the course you need:
 - At least 5 points per deliverable
 - At least 27 points in total from the deliverables
- Your work should be documented via version control (Git and GitHub)
- For deliverables you submit:
 - A Jupyter notebook containing the code and a discussion of your results
 - A pdf version of this notebook
 - A description of GenAI use, for example a summary of the chat log as you have done earlier
 - A link to the GitHub repository for the project

- A short (5 min) video discussing your code
- Note that the usual NMBU rules for plagiarism and references also apply here
 - You must hand in your own code!
 - If you take ideas from elsewhere, this must be described (for example by a link to the webpage you got the code from, or described in your GenAI log)
 - Other sources used should also be referenced
 - We have not done plagiarism control these first weeks, but suspected plagiarism will be flagged in Canvas for the deliverables

Fallback deliverable

- Given in week 49 (the last week of this course)
- For those missing points from the first four (or now five) weeks of exercises
- For those missing some points from the deliverables
 - To get this you must have gotten at least 2 points per deliverable and at least 20 points in total from the deliverables

Jupyter Notebook

- You will use this for your deliverables
- Jupyter allows you to combine code, code output and nicely formatted text (markdown) in the same document
- The document you are reading right now is a Jupyter notebook
- File extension ipynb, but under the hood it is a json file
 - Jupyter used to be called IPython Notebook, therefore ipynb

How to set up in VS Code

- [Guide for getting started](#)
- Get the Jupyter extension in VS Code
- Install the relevant Python packages: `python -m pip install jupyter`
- Create your first notebook:
 - Open the command palette (`Ctrl+Shift+P` or `Cmd+Shift+P`) and select `Create: New Jupyter Notebook`
 - Select your venv as kernel (upper right corner in VS Code)
 - Start writing by adding Code or Markdown cells
- [Guide to Markdown syntax](#)
- VS Code has an Export option, that allows you to export the notebook to html or pdf.
 - The pdf converter requires TeX, and can sometimes be problematic to get to work
 - The easier option is to export to html, open this in a browser and print to pdf there instead
 - You might need to specifically install the nbconvert package: `python -m pip install nbconvert`

- Can't get the html export to work either? Try this instead (in the VS Code terminal):

```
python -m nbconvert --to html path\to\your_file.ipynb
```

Collaborative workflows using GitHub

- See [this chapter](#) from the ProGit book, in particular the "Private Small Team" section, which is the most relevant one for INF201

Recommended workflow for INF201 deliverables

1. Creating project repository

- One of you creates the repository in GitHub (webpage), and will be the host for the project
- Set up the repository like we discussed last week
- In GitHub, go to Settings - Collaborators - Add people
- Search for the other collaborator(s) by their GitHub usernames or emails and send invitations
- They accept the invitation to get write permission to the repository
- Once everyone has accepted the invitation, everyone can clone the repository to their laptop
- Note: For small group projects like this, I don't recommend forking. Forking creates disconnected repositories that makes everyday collaboration more difficult.

2. Protect `main` and use branches well

- `main` is (in general) supposed to represent working runnable code
- It is therefore generally a bad idea to push directly to main with `git push origin main`
- Use branches for development, and merge them to main when they are working well
- It is a good idea to have a naming convention for the branches, to make it clear what they are about. Some examples:
 - `fix/file-save-bug` for bug fixes
 - `feature/csv-reader` for developing a new feature in the project

3. Working on a branches and creating pull requests

- You publish the branch you have created by clicking "Publish Branch" in VS Code or in terminal by typing `git push -u origin <branch>`
- When you go to GitHub, there will now be a pop-up asking you to "Compare & pull requests"
- Clicking this will allow you to review the changes in this branch, and tell you if this can successfully be merged with `main`
- If you want to do more work on the branch before merging with `main`, click the "Create Pull Request" button and select "Create Draft Pull Request"
- Keep working and commit and push often.

- When the branch is ready, click the "Ready for review" button at the bottom of the draft pull request
4. Review pull requests
 - If you have assign a collaborator to review the code, they will get a notification about this
 - Note: This should be done by someone who did not write the code!
 - This person then looks over the changes and decide if the merge should be approved, or if changes should be done
 - If the pull request is approved, you can "Merge pull request" and "Confirm merge", which will merge the branch into `main`
 - After the merge, the branch can be deleted to keep the repository tidy
 - See the attached pull request template (generated using Gemini)
 5. After merging into main, all collaborators should pull to their own laptop
 - Switch to `main` if you are not already there: `git switch main` (or select main in VS Code)
 - Select "Pull" from the VS Code menus, or on terminal: `git pull`
 - Delete your local version of the branch that has been removed from the origin: `git branch -d <branch-name>`

Collecting data from external sources

- We will specifically talk about getting data through [APIs](#) (application programming interfaces)
- Many organisations provide data for use on the web
- Some are very open, e.g., The Norwegian Meteorological Institute
 - Only registration required
- Others are more restrictive, e.g., the Norwegian Road Authority (Statens Vegvesen)
 - Application required, access from specific IP address only
- Yet others require secure authentication, e.g., Canvas
- But general principles are the same for a very wide variety of services

Representational State Transfer (REST)

- [REST](#) is a widely used technique for interaction between clients (web browsers, programs using data) and servers
- It is a stateless protocol: the server handles each request independently
- The server offers data through a Web Application Programming Interface (API), i.e., a set of rules describing how to request data and in which form data will be returned
- Data is typically request by a GET request to the server sending a Uniform Resource Identifier (URI), a URL with additional parts to specify which data are requested
 - Example: https://frost.met.no/observations/v0.jsonld?sources=SN17850&elements=mean%28air_temperature+P1D%29&referencetime=08-01%2F2021-09-22
 - The same with line breaks for clarity:
`https://frost.met.no/observations/v0.jsonld
?sources=SN1785`

```
&elements=mean%28air_temperature+P1D%29
&referencetime=2021-08-01%2F2021-09-22'
```

- Server returns data, most often in JSON format, but HTML or XML are also used
- In Python, we use the package `requests` for communicating with APIs

Obtaining meteorological data from frost.met.no

- For your first deliverable, you will work with data the Norwegian Meteorological Service
- You must register for a client-ID to access their services (only email required)
 - See <https://frost.met.no/auth/requestCredentials.html>
 - We will only use open services, none that require OAUTH2 authentication
- See the following information pages for more information about frost and their services:
 - <https://frost.met.no/index.html>
 - <https://frost.met.no/termsofuse2.html>
 - <https://frost.met.no/howto.html>
 - <https://frost.met.no/api.html#/observations>
- The following example is based on https://frost.met.no/python_example.html

Getting and analysing temperature data

- We will be using `requests` for getting the data and `pandas` for working with it

```
In [32]: import pandas as pd
import requests
```

- I first read my client-ID from a file
- See this weeks deliverable for tips on how to manage IDs like this, that are linked to your name

```
In [33]: client_id = open('../.../frost_credentials.txt').read().strip()
```

- To request data from Frost we need to define
 - where we want to send the request (this is the same for all requests to Frost)
 - what information we want
- In this example we will request the following data
 - From Ås (station ID: SN17850)
 - Daily (PD1) mean air temperature
 - For all of January 2026

```
In [34]: endpoint = 'https://frost.met.no/observations/v0.jsonld'
parameters = {
    'sources': 'SN17850',
    'elements': 'mean(air_temperature P1D)',
```

```
'referencetime': '2026-01-01/2026-01-31',
}
```

- The request it sent using the `get` function from the `requests` package
- This will return a status code we can use to check that all went well (200 means all is good)

```
In [35]: frost_response = requests.get(endpoint, parameters, auth=(client_id, ""))
assert frost_response.status_code == 200
```

- Let's have look at `frost_response` by examining the header information

```
In [36]: for header_name, header_value in frost_response.headers.items():
print(f'{header_name:16s}: {header_value}')
```

```
x-frost      : cache-status=hits:0/misses:0/noncacheable:1
cache-control : max-age=-1
date          : Sun, 04 Oct 2026 09:06:13 GMT
content-type  : application/json
content-length : 26767
```

- The data is supplied in json form
- We extract this, which in Python will be a dict

```
In [37]: frost_payload = frost_response.json()
type(frost_payload)
```

```
Out[37]: dict
```

```
In [38]: frost_payload.keys()
```

```
Out[38]: dict_keys(['@context', '@type', 'apiVersion', 'license', 'createdAt', 'queryTime', 'currentItemCount', 'itemsPerPage', 'offset', 'totalItemCount', 'currentLink', 'data'])
```

```
In [39]: for field_name, field_value in frost_payload.items():
if field_name == 'data':
continue
print(f'{field_name:16s}: {field_value}')
```

```
@context      : https://frost.met.no/schema
@type         : ObservationResponse
apiVersion    : v0
license       : https://creativecommons.org/licenses/by/3.0/no/
createdAt     : 2026-10-04T09:06:12Z
queryTime     : 0.309
currentItemCount: 30
itemsPerPage  : 30
offset        : 0
totalItemCount : 30
currentLink   : https://frost.met.no/observations/v0.jsonld?sources=SN17850&elements=mean%28air_temperature+P1D%29&referencetime=2026-01-01%2F2026-01-31
```

- Note the license here: [Creative Commons Byline v3.0](https://creativecommons.org/licenses/by/3.0/no/)

- This means that we are quite free to use the data *as long as we credit the Norwegian Meteorological Institute as source*

```
In [49]: frost_payload['data'][0]
```

```
Out[49]: {'sourceId': 'SN17850:0',
  'referenceTime': '2026-01-01T00:00:00.000Z',
  'observations': [{'elementId': 'mean(air_temperature P1D)',
    'value': -0.8,
    'unit': 'degC',
    'level': {'levelType': 'height_above_ground', 'unit': 'm', 'value': 2},
    'timeOffset': 'PT0H',
    'timeResolution': 'P1D',
    'timeSeriesId': 0,
    'performanceCategory': 'C',
    'exposureCategory': '1',
    'qualityCode': 0},
    {'elementId': 'mean(air_temperature P1D)',
    'value': -1.9,
    'unit': 'degC',
    'level': {'levelType': 'height_above_ground', 'unit': 'm', 'value': 2},
    'timeOffset': 'PT6H',
    'timeResolution': 'P1D',
    'timeSeriesId': 0,
    'performanceCategory': 'C',
    'exposureCategory': '1'}]}
```

- We have two entries, which are dictionaries
 - Each entry starts with sourceId
 - Each entry contains an observations array with two dictionaries
 - Averages midnight-to-midnight ('timeOffset': 'PT0H')
 - Averages morning-to-morning ('timeOffset': 'PT6H')
 - We want only the first observation here
 - For each observation, we want the value entry
- We use a list comprehension to extract just the data we want
 - Time entries are converted to Pandas Timestamp objects

```
In [ ]: temp_data = [{'Time': pd.to_datetime(entry['referenceTime']),
  'T_avg': entry['observations'][0]['value']}
  for entry in frost_payload['data']]
temp_data[:4]
```

```
Out [ ]: [{'Time': Timestamp('2026-01-01 00:00:00+0000', tz='UTC'), 'T_avg': -0.8},
  {'Time': Timestamp('2026-01-02 00:00:00+0000', tz='UTC'), 'T_avg': -5.6},
  {'Time': Timestamp('2026-01-03 00:00:00+0000', tz='UTC'), 'T_avg': -7.3},
  {'Time': Timestamp('2026-01-04 00:00:00+0000', tz='UTC'), 'T_avg': -7.2}]
```

```
In [47]: T_data = pd.DataFrame.from_records(temp_data).set_index('Time')
T_data
```

Out[47]:

	T_avg
Time	
2026-01-01 00:00:00+00:00	-0.8
2026-01-02 00:00:00+00:00	-5.6
2026-01-03 00:00:00+00:00	-7.3
2026-01-04 00:00:00+00:00	-7.2
2026-01-05 00:00:00+00:00	-7.9
2026-01-06 00:00:00+00:00	-10.5
2026-01-07 00:00:00+00:00	-11.2
2026-01-08 00:00:00+00:00	-12.0
2026-01-09 00:00:00+00:00	-5.7
2026-01-10 00:00:00+00:00	-13.0
2026-01-11 00:00:00+00:00	-17.1
2026-01-12 00:00:00+00:00	-10.4
2026-01-13 00:00:00+00:00	-3.1
2026-01-14 00:00:00+00:00	-1.6
2026-01-15 00:00:00+00:00	-1.9
2026-01-16 00:00:00+00:00	3.0
2026-01-17 00:00:00+00:00	0.9
2026-01-18 00:00:00+00:00	0.9
2026-01-19 00:00:00+00:00	-0.1
2026-01-20 00:00:00+00:00	-2.7
2026-01-21 00:00:00+00:00	-2.2
2026-01-22 00:00:00+00:00	-1.9
2026-01-23 00:00:00+00:00	-4.1
2026-01-24 00:00:00+00:00	-4.5
2026-01-25 00:00:00+00:00	-4.9
2026-01-26 00:00:00+00:00	-8.6
2026-01-27 00:00:00+00:00	-7.1
2026-01-28 00:00:00+00:00	-7.2
2026-01-29 00:00:00+00:00	-9.5
2026-01-30 00:00:00+00:00	-5.8

Recursive functions

- A **recursive function** is a function that calls itself
- This breaks the problem down into smaller steps
- A recursive functions needs to parts:
 - Base case: The stopping condition that returns a value without making further recursive calls. Without this the keep just keep running without knowing when to stop
 - Recursive step: The part where the function calls itself with an altered argument moving it closer to the base case.
- Example: Calculating factorials
 - $n! = n * (n - 1) * (n - 2) * \dots * 2 * 1$
 - We use the fact that $n! = n * (n - 1)!$ to create a recursive function

```
In [48]: def factorial(n: int) -> int:
          """Calculate the factorial of n"""
          # Base case: stop when n is 1 or less
          if n <= 1:
              return 1

          # Recursive step: call factorial with (n - 1)
          return n * factorial(n - 1)

print(factorial(5)) # Output: 120
```

120

Character encoding

- How does the computer view a string or the contents of a file?

ASCII

- American Standard Code for Information Interchange
- From the 1960s
- 7 bit, numerical values 0-127

Range	Hex	Values	Description
0-31	0x00-0x1f	Control characters, e.g., "newline"	
32	0x20	Space	
33-47	0x21-0x2f	Punctuation	
48-57	0x30-0x39	0-9	
58-64	0x3a-0x40	Punctuation	
65-90	0x41-0x5a	A-Z	
91-96	0x5b-0x60	Punctuation	
97-122	0x61-0x7a	a-z	
123-126	0x7b-0x7e	Punctuation	
127	0x7f	Control character DEL	

Unicode UTF-8

- ISO 10646 standard
- Provides 1,114,112 code points which can represent one symbol each
- Currently, 159,801 code points defined
 - 154 scripts/languages
 - mathematics

- emojis, ...
- Code points up to 255 (0xff) agree with ASCII (to 127) and [ISO 8859-1 Latin 1](#) for 128-255
- Code points are abstract numbers
- Code points are written as U+005a (capital "Z")
- Actual storage: Unicode Transformation Format (UTF)
 - Most common: UTF-8 (almost 99% of all websites)
 - UTF-8 is default for Python
 - Also used: UTF-16 and UTF-32
 - UTF-8 encodes the number of a Unicode code point in 1, 2, 3 or 4 bytes
 - UTF-8 is identical to ASCII up to 127
 - Number of bytes and bits for different code point (CP) ranges (after [Wikipedia](#)):

Bytes	Bits for CP	First CP	Last CP	Byte 1	Byte 2	Byte 3	Byte 4	-: -: -: -: -: -: -: -:
1	7	U+0000	U+007F	0xxxxxxx				
2	11	U+0080	U+07FF	110xxxxx	10xxxxxx			
3	16	U+0800	U+FFFF	1110xxxx	10xxxxxx	10xxxxxx		
4	21	U+10000	U+10FFFF	11110xxx	10xxxxxx	10xxxxxx	10xxxxxx	

Character encoding in Python

- `str` data type fully supports Unicode
- Each character in a string is represented by its Unicode code point
 - `chr()` converts code point to single-character string
 - `ord()` converts single-character string to code point

```
In [50]: cps = [ord(c) for c in ['A', 'a', '0', 'æ', 'α', '€', 'サ', '🤪']]
cps
```

```
Out[50]: [65, 97, 48, 230, 945, 8364, 12469, 128514]
```

```
In [51]: [hex(cp) for cp in cps]
```

```
Out[51]: ['0x41', '0x61', '0x30', '0xe6', '0x3b1', '0x20ac', '0x30b5', '0x1f602']
```

- Have a look back at the week 36 exercises where we used this to print the alphabet
- These code point values are used when sorting strings

How do I know which encoding a file has?

- This is more difficult than it sounds!
- Plain text files do not always contain built-in metadata stating their encoding.
- They will sometimes have a [Byte Order Mark \(BOM\)](#)

- A specific sequence of bytes placed at the very beginning of a file to signal its Unicode encoding and byte order (endianness)
- Some of the most common BOMs:

[Encoding]	BOM (hex)	Python Byte String	[-:]-[:]-[:]	UTF-8
UTF-8	EF BB BF	<code>b"\xef\xbb\xbf"</code>		
UTF-16 (little-endian)	FF FE	<code>b"\xff\xfe"</code>		
UTF-16 (big-endian)	FE FF	<code>b"\xfe\xff"</code>		

- When trying to figure out what encoding a file has, you can read the file in byte mode and look for a BOM

```
In [53]: from pathlib import Path

file_path = Path("file_utf8_with_bom.txt")
raw_bytes = file_path.read_bytes()
raw_bytes
```

```
Out[53]: b'\xef\xbb\xbfThis is a one-line file.'
```

- This file has the UTF-8 BOM
- Now, let's look at a similar file without a BOM

```
In [54]: file_path = Path("file_utf8_no_bom.txt")
raw_bytes = file_path.read_bytes()
raw_bytes
```

```
Out[54]: b'This is a one-line file.'
```

- Suggested method for identifying the encoding of a file:
 1. Check for BOM: If present, read the file with encoding `UTF-8-sig` or `UTF-16` (or `UTF-32`)
 2. Attempt to read the file with UTF-8 encoding
 3. If this fails (`UnicodeDecodeError`), try reading it with `cp1252` or Latin-1 encoding
 - `cp1252` is almost identical to Latin-1, and favoured by Windows
- The Python package [Charset normalizer](#) will analyse the file and predict the character encoding
- Even if Unicode in the form of UTF-8 is the most common system, there are a lot of different character encoding systems
 - [This is an overview of all encodings supported in Python](#)

```
In [62]: from charset_normalizer import from_path

results = from_path("file_utf8_no_bom.txt")
best_match = results.best()
print("Detected Encoding:", best_match.encoding)
```

```
Detected Encoding: ascii
```

- Why did this say ascii and not UTF-8?
- Most likely because the file is very small, and therefore don't have enough content to make a good guess
- What if we instead check the file containing a BOM?

```
In [63]: results = from_path("file_utf8_with_bom.txt")
best_match = results.best()
print("Detected Encoding:", best_match.encoding)
```

Detected Encoding: utf_8

Encoding and decoding

- Encoding: converting a str to its bytes representation
- Decoding: Converting from some encoding to Python `str`
- Strange things can happen if we choose the wrong encoding to decode with
- UTF-8 is default encoding

```
In [64]: text = "NMBU er i Ås kommune."
text_utf8 = bytes(text, encoding='utf-8')
text_latin1 = bytes(text, encoding='latin1')
print('UTF-8 :', text_utf8)
print('Latin-1:', text_latin1)
```

UTF-8 : b'NMBU er i \xc3\x85s kommune.'

Latin-1: b'NMBU er i \xc5s kommune.'

- Note the different encodings: UTF-8 requires two bytes for each Norwegian character
- Now, let's try to decode this again

```
In [65]: text_utf8.decode()
```

Out[65]: 'NMBU er i Ås kommune.'

- As UTF-8 is the default, this worked well
- What about the Latin-1 encoded string?

```
In [66]: text_latin1.decode()
```

```
-----
UnicodeDecodeError                                Traceback (most recent call last)
Cell In[66], line 1
----> 1 text_latin1.decode()

UnicodeDecodeError: 'utf-8' codec can't decode byte 0xc5 in position 10: invalid continuation byte
```

- We can tell `decode()` how to handle errors

```
In [67]: text_latin1.decode(errors='replace')
```

Out[67]: 'NMBU er i Ås kommune.'

```
In [68]: text_latin1.decode(errors='ignore')
```

Out[68]: 'NMBU er i s kommune.'

```
In [69]: text_latin1.decode(encoding='latin1')
```

Out[69]: 'NMBU er i Ås kommune.'

```
In [70]: text_utf8.decode(encoding='latin1')
```

Out[70]: 'NMBU er i Å\x85s kommune.'

- Most Python readers will have encoding as a keyword argument